

This configuration sets up a simple round-robin load balancer that forwards requests to three backend servers.

Advanced load balancing techniques

To create a more sophisticated load balancer, you can implement various load balancing methods provided by NGINX.

Round robin: Requests are distributed evenly across the servers, with the option of weight servers.

Least connections: Requests are sent to the server with the fewest active connections.

IP hash: Client IP addresses are hashed and mapped to servers to ensure a client consistently reaches the same server (useful for session persistence).

Here's how you can configure a least connections load balancer:

```
nginx
upstream myapp {
    least_conn;
    server server1.example.com weight=3;
    server server2.example.com;
    server server3.example.com;
}
```

In this configuration, *server1.example.com* is assigned a higher weight, meaning it will receive more connections than the others.

Health checks and failover

To ensure high availability, NGINX can be configured to perform health checks on backend servers. If a server fails a health check, NGINX will stop sending traffic to it until it becomes healthy again.

In NGINX Plus, active health checks can be configured directly. For the open source version of NGINX, you can use third-party modules or rely on passive health checks, which remove a server from the pool if a connection fails.

Session persistence

Certain applications require that a user's session remains on the same backend server. This can be achieved with the *ip_hash* directive or, in NGINX Plus, with the *sticky* directive.

SSL termination

NGINX can handle SSL termination, decrypting HTTPS traffic before passing it on to the backend servers. This reduces the load on the backend servers and centralises SSL management. To configure SSL termination, you need to include the *ssl_certificate* and *ssl_certificate_key* directives in your server block and listen on port 443.

Advanced configuration options

NGINX offers a wide range of configuration options that allow you to optimise the behaviour of your load balancer.

For example, you can adjust buffer sizes, timeouts, and the maximum number of connections. You can also use the *proxy_cache* directive to cache responses from your backend servers, reducing load and improving response times.

Security features

Security is a critical aspect of load balancing. NGINX provides several features to enhance security, such as rate limiting, which can protect against DDoS attacks, and the ability to block traffic based on IP addresses or other request attributes.

Monitoring and logging

To maintain a healthy load balancer, you need to implement monitoring and logging. NGINX writes logs to */var/log/nginx/access.log* and */var/log/nginx/error.log* by default. These logs can be analysed to understand traffic patterns and identify issues. For more detailed monitoring, you can use the NGINX status module or integrate with third-party monitoring tools.

Testing and deployment

Before deploying your load balancer, it's essential to test it thoroughly. This includes testing for performance, failover, and handling of various traffic patterns. Once you're confident in the setup, you can deploy it to production.

Ongoing management

After deployment, you'll need to manage your load balancer by keeping NGINX and the operating system up to date, adjusting configurations as needed, and monitoring the system's health.

Scaling and high availability

As your traffic grows, you may need to scale your load balancer horizontally by adding more NGINX instances. You can also set up a high-availability pair using keepalived or similar tools to ensure that if one load balancer fails, another takes over without service interruption.

Building an advanced NGINX load balancer requires careful planning, configuration, and management. By following these steps and considering your specific requirements, you can create a load balancer that is efficient, secure, and highly available. Remember to consult the NGINX documentation for detailed instructions and best practices specific to your use case. 

Reference

- <https://nginx.org/>

By: Neetesh Mehrotra

The author is working in EY as a senior. His areas of interest are Java development and automation testing.